Assignment No. 10

- **TITLE –** In calculators converting and evaluating arithmetic expressions accurately is essential. Use stack data structures to convert infix expressions to postfix and prefix formats, and then evaluate the postfix expression. Ensure your implementation handles parentheses, multi-digit operands, and input errors gracefully.

- **THEORY –**

Arithmetic expressions can be written in three notations:

1. Infix – Operators between operands

Example: A + B * C

2. Postfix (Reverse Polish Notation) – Operators after operands

Example: A B C * +

3. Prefix (Polish Notation) – Operators before operands

Example: + A * B C

Computers prefer postfix and prefix because they remove the need for parentheses and operator precedence rules.

A stack is a LIFO (Last-In First-Out) data structure.

→ It is ideal for expression processing because operators must be stored temporarily until they can be applied.

→ Stacks help in:

- Managing operators
- Handling parentheses
- Preserving precedence (* before +)
- Reversing order for prefix conversion
- Evaluating postfix by storing intermediate results

- Infix to Postfix Conversion (Shunting-Yard Method)

To convert infix to postfix:

1. Scan the infix expression from left to right
2. Operands → add directly to postfix output
3. Operators → push to stack
   Pop operators from stack if they have higher or equal precedence
4. Left parenthesis '(' → push to stack
5. Right parenthesis ')' → pop until '(' is found
6. After scanning → pop all remaining operators to output

This removes the need for parentheses and produces a postfix expression.

- **Infix to Prefix Conversion**

Prefix can be generated using:

1. **Reverse the infix expression**
2. Swap (with)
3. Convert reversed expression to postfix

4. Reverse the postfix result → gets **prefix**

This method works because reversing changes order of operations in a predictable way.

- **CODE –**

```
#include <bits/stdc++.h>
using namespace std;
bool isOperator(char c){
    return (!isalpha(c) && !isdigit(c));
}
int getPriority(char C){
    if(C=='-' || C=='+'){
        return 1;
    }
    else if(C=='*' || C=='/'){
        return 2;
    }
    else if(C=='^'){
        return 3;
    }
    return 0;
}
string infixToPostfix(string infix){
    infix = '(' + infix + ')';
    int l = infix.length();
    stack<char> charStack;
    string output;
    for(int i = 0;i < l;i++){
        if(isalpha(infix[i])||isdigit(infix[i])){
            output+=infix[i];
        }
        else if(infix[i]=='('){
            charStack.push('(');
        }
        else if(infix[i]==')'){
            while(charStack.top()!='('){
                output+=charStack.top();
                charStack.pop();
            }
            charStack.pop();
        }
        else{
            if(isOperator(charStack.top())){
                if(infix[i]=='^'){
                    while(getPriority(infix[i])<=getPriority(charStack.top())){
                    output+=charStack.top();
                    charStack.pop();
                    }
                }else{
                    while(getPriority(infix[i])<getPriority(charStack.top())){
                    output+=charStack.top();
                    charStack.pop();
                    }
                }
                charStack.push(infix[i]);
            }
        }
```

```
    }
    return output;
}
string infixToPrefix(string infix){
    int l = infix.length();
    reverse(infix.begin(),infix.end());
    for(int i=0;i<l;i++){
        if(infix[i]=='('){
            infix[i]=')';
        }
        else if(infix[i]==')'){
            infix[i]='(';
            i++;
        }
        }
    string prefix = infixToPostfix(infix);
    reverse(prefix.begin(),prefix.end());
    return prefix;
    }
int evaluatePostfix(string postfix){
    stack<int> s1;
    int l = postfix.length();
    for(int i=0;i<l;i++){
        if(isdigit(postfix[i])){
            s1.push(postfix[i]-'0');
        }
        else{
            int val1 = s1.top();
            s1.pop();
            int val2 = s1.top();
            s1.pop();
            switch(postfix[i]){
                case '+':
                    s1.push(val2+val1);
                    break;
                case '-':
                    s1.push(val2-val1);
                    break;
                case '*':
                    s1.push(val2*val1);
                    break;
                case '/':
                    s1.push(val2/val1);
                    break;
            }
        }
    }
    return s1.top();
}
int main(){
    string infix;
    cout<<"Enter Infix Expression: ";
    getline(cin,infix);
    cout<<"Postfix Expression: "<<infixToPostfix(infix)<<endl;
    cout<<"Prefix Expression: "<<infixToPrefix(infix)<<endl;
    cout<<"Postfix Evaluation: "<<evaluatePostfix(infixToPostfix(infix))<<endl;
    return 0;
}
```

- **OUTPUT –**

  Enter Infix Expression: (9+6/7)+(8*2)/(16-3)
  Postfix Expression: 967/+82*163-/+
  Prefix Expression: ++9/67/*82-163
  Postfix Evaluation: 16

- **CONCLUSION –**

  This practical lab demonstrates how stack is used to implement infix to postfix and infix to prefix conversions with the evaluation of infix expressions.